

ER-301 Sequencer — Complete Specification

Identity & scope

A discrete-event pattern instrument that lives alongside the chain (which remains the continuous-signal instrument). The sequencer outputs as virtual mod sources in the existing source picker (`seq.*` namespace, alongside `sc.cv`, `sc.tr`, `mic.*`), consumed by chain destinations like any other typed source. No new abstraction is introduced at the chain interface — the sequencer is a producer of typed values, the chain is what does anything signal-flavored with those values.

The sequencer is **firmware-universal**: it exists on both the desktop unit and the portable variant. Only the takeover-mode UI differs (portable gets a dedicated takeover view; desktop edits the sequencer unit inline like any other unit).

The sequencer is implemented as a **chain unit type**. Multiple sequencer instances can run in parallel within a patch via standard unit instantiation. Each instance has its own L1/L2 state. Polyphony, multi-voice patterns, and parallel rule streams are all achieved by running multiple sequencer units, not by one sequencer becoming multi-voice internally.

Architectural principles

The discipline that holds the design together:

- **No general math.** Operations are typed, local to columns, drawn from a fixed vocabulary. There is no arbitrary value math across heterogeneous types.
- **No continuous-time semantics.** Everything evaluates at master ticks. Audio-rate or signal-rate modulation is the chain's job, not the sequencer's.
- **No smoothing or slew on outputs.** Outputs are stepped. Smoothing happens chain-side via slew units consuming `seq.*` sources.
- **Cross-column value dependency only via predicates.** Column B's *value* never depends arithmetically on column A. Column B's *behavior* (firing, jumping, modifying) can depend on column A through predicates.
- **No FX kitchen sink.** All conditional and structural behavior lives in L2 as `predicate:action` cells. There is no separate FX column.

- **Destructive writes from L2 to L1.** L2 actions write to L1 with the same authority as user edits. There is no distinction between authored and mutated state.
- **Single-pass tick evaluation.** Rules don't observe each other's effects within the same tick. Cascading happens across ticks, not within them.

Two-layer model

L1 — Substrate

Six columns of typed literal values. Pure data, no behavior. The columns are fixed in role:

Column	Type	Role
1	CV	Free CV (typically pitch in users' patches)
2	CV	Free CV (typically secondary modulation target)
3	CV	Free CV (typically tertiary)
4	Time	Gate length
5	CV	Gate amplitude
6	Time	Step length

The three CV columns are typed-but-generic — they emit normalized CV values, with downstream destinations interpreting them in their native type domain (pitch, filter cutoff, sample position, etc.). Type-coherence across the entire 301 isn't enforceable at the sequencer level; the sequencer treats CV as a single domain, and chain units interpret as needed.

Each L1 column has its own length and loop position. All columns advance simultaneously on the master tick; polymeter is achieved by columns having different lengths. A column of length 12 plays alongside a column of length 16 produces a 48-tick combined cycle (LCM).

Each L1 cell holds an explicit literal value. Empty cells emit the column's typed zero. There is no hold glyph and no special cell states. Sustained values are achieved through the **shift+encoder fill** gesture (see Authoring Gestures below).

L2 — Rules

Six columns of sparse `predicate:action` cells. Each L2 cell is **always implicitly IF**: the cell exists only as a conditional rule. There is no value/rule mode toggle per cell.

L2 columns have independent lengths, separate from L1 column lengths. An L2 column of length 7 produces a 7-step rule cycle, looping independently of any L1 column. Predicates evaluate locally — %4 means “every 4th pass through this cell,” counted by the cell’s own pass counter, not by master step position.

L2 columns are not paired 1:1 with L1 columns by purpose. Any L2 cell can target any L1 column. The user organizes rules across L2 columns by their own intent (e.g., “L2 column 1 holds fills and jumps, L2 column 2 holds probability gates”); the architecture does not impose a relationship.

Master clock

Step length (L1 column 6) is the master clock per step. The value at the step length column’s current playhead position determines how long the master tick spends on the current step before advancing.

Step length is itself a column with its own length, so the rhythmic feel of the pattern is a loopable, polymetric layer. A step length sequence of $[1/4, 1/8, 1/8, 1/4, 1/8, 1/16, 1/16, 1/4]$ looping every 8 steps produces a recurring rhythmic feel that other columns play polymetrically over.

Step length values are expressed as fractions of beats relative to a global tempo (tempo set elsewhere — admin menu, MIDI clock, or chain-side clock source). External clock sync is supported via the existing 301 clock infrastructure; default is internal BPM.

Gate behavior

Gate firing semantics:

- Gate column (L1 column for gates — implementation detail; functionally one of the six columns or a derived attribute of CV1) fires at the start of each step where its cell is non-zero.
- Gate length (L1 column 4) determines how long the gate stays high after firing.
- Gate amplitude (L1 column 5) determines the gate’s amplitude when high (for destinations that consume it as continuous level rather than just on/off).

When **gate length > step length**: the gate stays high through the step boundary (tie/legato semantics). CV columns update at the step boundary regardless — emitted CV values reflect the new step’s data, but the gate remains held until its length elapses. This produces sliding/legato lines naturally; users wanting held-pitch ties achieve this by holding the same CV value across steps.

When **gate length** $\leq$ **step length**: standard articulation. Gate fires, holds for gate length, releases. Remainder of step is silent.

Gate length and gate amp values are only consulted on steps where the gate fires; values at non-firing steps are inert. Users may leave them at column defaults without consequence.

Predicate vocabulary

Predicates evaluate to booleans. Two structural categories:

Local predicates (stateful, per-cell counter):

- `%N` — every N passes through this cell
- `?P` — probability P (0–100) evaluated each pass
- `=N` — exactly the Nth pass (then never again until pattern restart)
- `~N` — every pass after the Nth (warmup gating)

Reading predicates (stateless, snapshot of current state):

- `A>B` , `A<B` , `A=B` — column comparisons (where A, B are column letters)
- `A>literal` — threshold against a literal value (literal in column A's native type)
- `A!` — column A's gate is firing this tick
- `A=` — column A's value changed since previous tick
- `@a-b` — current step index in range a to b

Compounds: `&` (AND), `|` (OR), `!` prefix (NOT). One level of compounding only — no nested expressions beyond a single conjunction or disjunction. Compounds expand to a 7th editor slot temporarily when invoked.

State separation: stateful predicate counters update on every column-playhead pass through the cell, regardless of whether the predicate's truth value gates the action. So `?60` rolls every pass; `%4 & A>0.5` increments the modulo counter every pass and tests both conditions for firing.

Action vocabulary

Actions split into two categories by mutation behavior:

Stored-value actions (write destructively to L1 cell):

- $B+N$, $B-N$ — add/subtract literal N from column B at current step
- $B*N$, B/N — multiply/divide column B at current step
- $B=N$ — set column B at current step to literal N
- $B?$ — randomize column B at current step within its valid range

Transient actions (per-tick emit only, no L1 mutation):

- $B!$ — force column B's gate to fire this tick
- $B-$ — mute column B's emission this tick

Playhead actions (modify column position, no value mutation):

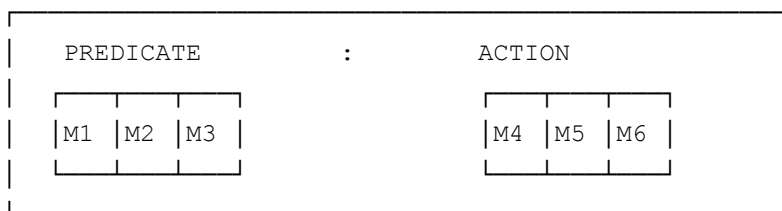
- B_n — bare number after column letter, jump column B's playhead to row n
- $*_n$ — global jump, all column playheads to row n
- $.n$ — self-jump, this column's playhead to row n (for rules acting on their own column)

Default action when no operator is specified: jump (e.g., $B9$ = "jump column B to row 9"). All other operations require an operator symbol.

Action target type-checking: operations must be valid for the target column's type. Time-typed columns (gate length, step length) accept add/scale/halve in time units. CV-typed columns accept add/scale/set in normalized CV space. Gate-typed actions (force-fire, mute) only apply to the gate column. The construction wizard only offers valid combinations.

Cell construction (stepped editor)

Six editor slots mapped to softkeys M1–M6, split as predicate-half and action-half with implicit `:` between:



M1: predicate type

M2: operand A

M3: operand B / compound

M4: action type

M5: target column

M6: action operand

Each softkey selects its corresponding slot for editing. Encoder rotates the selected slot's value (type-aware — rotates through valid options for the slot's role). Encoder click confirms and may auto-advance to next slot.

Shoulder buttons navigate slots (L = previous slot, R = next slot) for one-handed traversal without leaving the encoder.

Hard buttons in editor context:

- Home: exit editor without saving (revert cell to pre-edit state)
- Cancel: clear all slots back to defaults (reset cell to template)
- Up: jump to previous populated cell in this column
- Enter: commit cell, exit editor, advance playhead position to next step

Tooltip area (using available pixel real estate) shows the natural-language description of the currently selected slot plus a live preview of the cell's notation as it would appear in the grid. This serves both new users (learning glyphs from descriptions) and experts (terse glyph entry).

Type-aware operand display: literal operands render in the target column's native type. `+12` displays as `" +12 st"` or `" +1 oct"` or `" +C5"` depending on user setting when targeting a pitch-typed CV column. The same `+12` displays as `ms` or `beats` when targeting a time-typed column.

Default behavior on cell creation: insert a valid template (e.g., `%2:A!`) and let the user edit slots. First-use walkthrough available for new users; experienced users edit slots directly.

Compound predicates: holding M3 cycles through `(none) → AND → OR → NOT` modes.

AND/OR expand the cell into a temporary 7-slot editor for the second predicate clause. The cell renders with the compound expression in the L2 grid; truncation with continuation glyph `( ... )` for cells too long to display fully, with full notation visible on cell selection.

Authoring gestures

L1 cell editing: standard 301 idiom. Encoder click to enter edit mode, encoder rotation scrolls through values (type-aware), push-encoder for fine vs coarse rate, encoder click commits.

Bulk fill (shift + encoder scroll):

- User enters value at origin cell
- User holds shift modifier (typically right shoulder by default, user-assignable)

- User scrolls encoder up or down, range highlight extends from origin in the scroll direction
- Visual preview shows which cells will be affected, with new value shown overstriking any existing values
- User releases shift, fill commits aggressively (overwrites existing values in range)
- Range bounded by column loop length (no wrap)

This gesture is the primary way to author sustained values without per-cell repetition. Bidirectional (works scrolling either direction from origin).

L2 cell entry: navigate to empty L2 cell, encoder click to enter editor (described above). Empty L2 cells emit no rule firings.

Execution model

On every master tick:

1. Each L1 column's playhead advances by 1 within its own loop length
2. Each L1 column's current cell value is emitted to virtual outputs
3. Each L2 column's playhead advances by 1 within its own loop length
4. For each L2 column where the playhead lands on a populated cell:
 - Cell's pass counter increments (drives stateful predicates)
 - Predicate evaluates against tick-start state (snapshot at top of tick)
 - If predicate true, action fires
5. Actions apply in **grid-absolute order**: top-to-bottom, left-to-right across the L2 grid
6. Stored-value actions write destructively to L1 cells
7. Transient actions affect this tick's emission only
8. Playhead actions queue to take effect at tick boundary (next tick reads from new positions)
9. Gate state resolves (gate length vs step length determines whether previous gate is still held; tie/legato handled)
10. Final values emitted to chain via `seq.*` virtual outputs
11. Master tick scheduled for next step using current step length value

Single-pass discipline: predicates evaluate against tick-start state. If action A modifies a value mid-tick, action B firing later in the same tick still sees the original (tick-start) value when its predicate evaluates. Cascading reactivity happens across ticks, not within them. This is non-negotiable for predictable execution.

Action composition within a tick: stored-value actions compose by ordered application. If two actions both target column B at the current step (e.g., `+5` and `*2`), they apply in left-to-right grid order. Result is $(\text{value} + 5) * 2$. The user sees this composition order by reading L2 columns left-to-right.

Destructive writes

L2 actions write to L1 with the same authority as direct user edits. There is no separate "authored state" preserved alongside "current state"; L1 simply is the current pattern.

This means:

- A user opting to use L2 is opting into pattern evolution
- Patterns drift over time as rules execute
- The visible cell value in L1 is always the actual emitted value (no hidden mutation layer)
- Re-loading a pattern from a quicksave restores the L1 state at the moment of save

Patterns that should not evolve are kept L1-only (no L2 cells). The deterministic pattern composition use case is fully served by L1 alone.

Patterns that should evolve in bounded ways use L2 setter and jump actions to enforce structure: `A>0.9:A=0.5` keeps column A bounded; `%32:Bn` periodically resets column B's playhead. Bounded evolution is achieved through in-grammar discipline, not engine support for "authored value" reference.

RNG and reproducibility

The sequencer uses a **single global RNG seed per patch**. Seed is established at boot (or pattern initialization), persisted to quicksaves, and restored on reload.

This makes patterns reproducible: same patch, same quicksave, same evolution. A user sharing a quicksaved pattern with another user produces identical behavior on load.

Re-seeding is implicit on boot. Explicit re-seed via admin menu (deferred to UI implementation phase).

Quicksaves

Quicksaves are the sole preservation mechanism. A quicksave captures:

- L1 cell state (all columns, all rows, current values)
- L2 cell state (all rules, all positions)
- Column lengths and current playhead positions (L1 and L2)
- L2 cell pass counters
- RNG state (seed and current position)
- Tempo and timing context

A quicksave restored loads exact state — pattern continues from precisely where it was saved.

Snapshot/revert UX is **deferred to v2**. The leading candidate uses existing 301 preset machinery, presented in the sub-display. Final UI design awaits hardware availability for ergonomic testing on both the original 301 form factor and the portable's larger display, where space allocation differs.

Live performance affordances

Global L2 disable toggle: a control (likely the right slide switch's third position; subject to user-assignable layout) suspends all L2 rule execution while preserving current L1 state. Patterns continue playing as their current L1 values, but evolution stops. Re-enabling resumes from current state.

This serves multiple purposes:

- Live performance freeze (capture an interesting evolved state and hold it)
- Debugging during pattern authoring (observe current state without continued mutation)
- Returning to deterministic playback mid-session

Cross-platform implementation

The sequencer targets both AM335x (BeagleBone class) and CM4. Runtime cost per tick is bounded and modest:

- L1 emission: $O(\text{columns})$, trivial
- L2 evaluation: $O(\text{populated cells with playhead landing this tick})$, typically very small

- Predicate eval: bounded per-cell instruction count
- Action application: $O(\text{firing actions})$, small

Total per-tick cost is well under 100 microseconds on AM335x for typical patterns. The lower-spec target is not the constraint.

UI rendering is parameterized in terms of visible columns and visible rows rather than hardcoded pixel dimensions. The same grid logic renders to whatever pixel rectangle the platform provides. The original 301's display abstraction ($256 \times 64 + 128 \times 64$ logical) handles this via the existing display layer; sequencer rendering inherits the abstraction.

Column counts (6 L1 + 6 L2) and grammar (predicates, actions, operand types) are platform-invariant. Only the visible window over the underlying pattern varies.

Display layout

Layout is parameterized; specific rendering decisions defer to hardware availability. The general principle:

- L1 grid renders as a dense tracker-style grid: rows are steps, columns are L1 columns, each cell shows its current value
- L2 grid renders in the same shape, sparse (most cells empty)
- L1 and L2 are separate views with mode toggle (likely a slide switch position or hard key); inactive layer rendered as faded ghost behind active layer
- Active layer indicated by colored edge strip in chrome (warm hue for L1, cool hue for L2)
- Shared playhead across L1 and L2; moves through master step positions
- Per-column playheads visible as small markers on each column header (L1 and L2)

On the desktop unit ($256 \times 64 + 128 \times 64$), sequencer takeover uses the main display for L1/L2 grid and the sub display for cell editor or status. Limited rows visible at a time; scrolling follows playhead.

On the portable variant (640×480 IPS, $2 \times$ scaled to 320×240 logical), sequencer takeover uses the full panel for the grid. More rows visible simultaneously; the larger surface enables a tracker-style multi-screen view of the pattern.

Virtual outputs to chain

L1 column emissions surface in the source picker as:

- `seq.cv1` , `seq.cv2` , `seq.cv3` — stepped CV values from L1 columns 1–3
- `seq.gate` — gate signal (driven by gate column firing, modulated by gate length and amp)
- `seq.gatelen` — gate length value as readable scalar (rare use)
- `seq.steplen` — step length value as readable scalar (rare use)

These are stepped outputs. Smoothing is the chain's responsibility via slew units consuming these sources.

For multiple sequencer instances: each instance namespaces its outputs (e.g., `seq1.cv1` , `seq2.cv1`). Standard 301 unit instance naming applies.

Hard function assignment

The sequencer respects the device's user-assignable hard function placement (defined elsewhere in the device spec). Functions referenced by name in this spec — Shift, Coarse/Fine, Home, Cancel, Up, Enter, L1/L2 toggle — exist as fixed behaviors but their physical placement on the device is user-configurable.

Documentation and in-context UI hints reference functions by name, not by position. A user who has remapped Shift sees prompts referring to "Shift" and translates to wherever they have placed it.

What the sequencer does not do

For clarity:

- Does not process audio (chain's job)
- Does not modulate continuously (chain's job)
- Does not perform arbitrary value math across columns
- Does not have voice-oriented columns (no M8-style "this column is a synth voice")
- Does not reproduce chain functionality
- Does not have a separate FX column
- Does not implement song-mode pattern chaining (use long L2 patterns with conditional jumps, or arrange via DAW)
- Does not implement live MIDI/CV recording into L1 (potential v2 feature)

- Does not allow L2 cells to modify other L2 cells (rule definitions are static during playback)

Open items deferred to implementation

Items intentionally not specified, to be resolved during prototyping:

- Snapshot/revert UX (v2; awaits hardware for UI testing)
- Default hard function placement specifics (paper mockup phase)
- Whether playhead jumps reset their column's pass counters (resolved by feel during prototyping)
- Hold glyph for cells (rejected in favor of shift+scroll fill gesture)
- Live recording (potential v2)
- Pattern chaining / song mode (deferred to chain-side composition)
- Pattern variation/scene switching (chain-side unit concern)
- L2 meta-rules (out of scope for v1)

Reference card target

The full predicate and action vocabulary fits on a single printed reference card (4×6 inches estimated). Predicate vocabulary at ~7 entries plus ~4 compounds; action vocabulary at ~8 entries plus default-jump syntax. This is a deliberate scope constraint — additions to the grammar must justify their place on the card.

The card serves as marketing artifact, teaching tool, and durable reference. Keeping the grammar small enough to fit is part of the device's character.